

Database Lab Assignment: Milestones 2 to 5

Group Name: DesignDB Team **GitHub Repository:**

<https://github.com/horizon1122star/designdb/tree/Features>

System Workflow: DesignDB Architecture

The **DesignDB** platform operates as a multi-tier SaaS application designed to translate human natural language into fully functional, normalized relational databases and interactive Entity-Relationship Diagrams (ERDs).

1. How User Requirements Are Received

The user inputs their database requirements using a natural language prompt via the application's central text interface (e.g., "Build an e-commerce platform with users, products, orders, and reviews"). The Next.js frontend captures this input and sends it to our orchestration backend.

2. How Schemas Are Generated

The orchestration layer sends the prompt to an advanced Large Language Model (Groq / Llama 3) via an API Route. The LLM acts as an AI Database Architect, extracting entities, attributes, and relationships. It returns a strict, structured JSON payload representing the initial schema design.

3. How Normalization is Handled

Normalization is programmatically enforced in our execution scripts:

- **1NF (First Normal Form):** Ensures all attributes are atomic and single-valued.
- **2NF (Second Normal Form):** Ensures all non-key attributes are fully functionally dependent on the primary key (no partial dependencies).
- **3NF (Third Normal Form):** Resolves transitive dependencies, ensuring non-key attributes do not depend on other non-key attributes. If the AI output violates these forms (e.g., putting 'CustomerAddress' in an 'Order' table instead of linking it), the validation engine corrects it before returning the JSON payload.

4. How ER Diagrams Are Created

The validated JSON schema is parsed and transformed into **Mermaid.js** syntax by the backend. This syntax is passed back to the React frontend, where `dagre` and `@xyflow/react` (ReactFlow) map the tables, foreign keys, and columns into a dynamic, drag-and-drop 2D visual canvas.

5. How SQL Scripts Are Exported

Users can select their desired SQL dialect (PostgreSQL, MySQL, SQLite) in the settings. Our custom `export_sql` script translates the internal JSON representation into standard DDL (`CREATE TABLE` statements, `PRIMARY KEY` definitions, and `FOREIGN KEY` constraints), which the user can immediately download as a `.sql` file.

6. Data Flow from Input to Output

User Prompt → Groq API (JSON) → Normalization Engine → Prisma/PostgreSQL (Cloud Save) → ReactFlow (Visual ERD) → SQL/PNG Export

Milestone 2: Schema Normalization & ERD Updates

The internal database running **DesignDB** itself requires persistent storage for user projects. The schema consists of two core tables: `User` and `Project`.

Justification & Normalization (1NF to 3NF)

- **User Table:** Stores unique user identities (`id` , `email`). This is in 3NF because `email` depends directly and solely on the primary key `id` .
- **Project Table:** Stores ReactFlow canvas states.
 - **1NF:** All fields (`id` , `title` , `rawPrompt` , `nodesJson` , `edgesJson`) are atomic at the application level.
 - **2NF:** The primary key is a single column (`id`), eliminating partial dependency risks.
 - **3NF:** `userId` acts as a Foreign Key linking to `User` . There are no transitive dependencies (e.g., we do not store the User's email inside the Project table).

(Note: The ERD visualization is handled by the application itself and saved in the repository).

Milestone 3: Dataset Preparation & Dataflow Description

Dataflow Description

When a user clicks "**Save to Cloud**", the frontend extracts the exact visual state of the ERD (`nodes` and `edges`) using ReactFlow's internal hooks. This data is structured into a JSON payload and sent via a `POST` request to `/api/projects/save` .

The Next.js backend intercepts this request, uses the **Prisma ORM**, and performs an UPSERT operation on the `User` table (ensuring the user exists) followed by an INSERT into the `Project` table. The data is physically stored in a pooled **Supabase PostgreSQL** instance.

Synthetic Data (Dummy Data)

Because this is a schema generator, the "dataset" consists of JSON-serialized ReactFlow graphs. Dummy datasets were generated by interacting with the canvas, dragging tables, and saving them. We export CSVs programmatically from our Supabase dashboard to verify data ingestion.

Milestone 4: DDL Scripts (CREATE TABLE Statements)

The following DDL statements describe our persistent storage in Supabase:

```
```sql -- CreateTable: User CREATE TABLE "User" ( "id" TEXT NOT NULL, "email" TEXT NOT NULL, CONSTRAINT "User_pkey" PRIMARY KEY ("id") );

-- CreateIndex: User_email_key CREATE UNIQUE INDEX "User_email_key" ON "User" ("email");

-- CreateTable: Project CREATE TABLE "Project" ("id" TEXT NOT NULL, "userId" TEXT NOT NULL, "title" TEXT NOT NULL, "rawPrompt" TEXT, "nodesJson" JSONB NOT NULL, "edgesJson" JSONB NOT NULL, "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP, "updatedAt" TIMESTAMP(3) NOT NULL, CONSTRAINT "Project_pkey" PRIMARY KEY ("id"));

-- AddForeignKey ALTER TABLE "Project" ADD CONSTRAINT "Project_userId_fkey" FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE RESTRICT ON UPDATE CASCADE; ```
```

---

## Milestone 5: Database Population & Validation Queries

To simulate user activity and validate our deployment, we executed the following population and validation operations via Prisma and raw SQL:

### Data Ingestion (INSERT)

```
```sql INSERT INTO "User" ("id", "email") VALUES ('demo-user-id', 'demo@designdb.app');

INSERT INTO "Project" ("id", "userId", "title", "rawPrompt", "nodesJson", "edgesJson", "updatedAt") VALUES ('proj_123', 'demo-user-id', 'My First E-Commerce DB', 'E-commerce with users', '[]', '[]', NOW()); ```
```

Validation Queries

1. **Row Count Check:** ````sql SELECT COUNT(*) FROM "Project"; -- Verifies ingestion success ````
2. **Foreign Key Integrity Check:** ````sql SELECT p.id, p.title, u.email FROM "Project" p JOIN "User" u ON p."userId" = u.id; -- Ensures no orphaned projects exist without a valid user ````
3. **NULL Value Verification:** ````sql SELECT COUNT(*) FROM "Project" WHERE "nodesJson" IS NULL; -- Returns 0, verifying our NOT NULL constraints are active and ReactFlow states are securely saved. ````